DEPARTMENT OF COMPUTER ENGINEERING

ACADEMIC YEAR 2026–2027 | ODD SEMESTER | MINI PROJECT STAGE-1

MSME360: INTEGRATED WEB-BASED BUSINESS MANAGEMENT AND ANALYTICS PLATFORM FOR MSMEs

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Formal Technical Specification & Academic Baseline | Document Version 1.0 (Final)

Project & Academic Attribute	Specification Details
Project Title	MSME360: Integrated Web-Based Business Management & Analytics Platform
Document Identifier	MSME360-SRS-DOC-V1.0 (Draft for Technical & Academic Review)
Publication Date	October 19, 2026
Academic Group	Group No. 5 | Bachelor of Engineering (B.E.) — Computer Engineering (CE)
Curricular Mappings	DBMS, Design & Analysis of Algorithms (DAA), Software Engineering (SE), Web Tech (WT)
Faculty Project Guide	Prof. Jignesh Sisodia
Student Team Members	1. Aditya Malekar (UID: 2024300118) 2. Devank Kolpe (UID: 2024300129) 3. Harshil Mehta (UID: 2024300133)
Target Stack	Python 3.12, FastAPI, PostgreSQL, SQLAlchemy 2.0, React 18, Vite, Docker, Render, Vercel

Document Control, Approval Sign-Off & Status

Role / Reviewer	Name & Designation	Signature Placeholder	Date
Student Lead	Aditya Malekar (UID: 2024300118)	_____	19-Oct-2026
Student Member	Devank Kolpe (UID: 2024300129)	_____	19-Oct-2026
Student Member	Harshil Mehta (UID: 2024300133)	_____	19-Oct-2026
Faculty Project Guide	Prof. Jignesh Sisodia	_____	___-___-2026
Head of Department	Head, Dept. of Computer Engineering	_____	___-___-2026

Revision History

Version	Release Date	Author(s)	Status / Change Description
0.1	2026-08-15	Student Team (G5)	Initial requirements elicitation, domain boundary definition, and problem formulation.
0.5	2026-09-10	Student Team (G5)	Drafting of 3NF relational data model, RBAC policies, and core API service contracts.
1.0	2026-10-19	Student Team (G5)	Final comprehensive industrial-aligned SRS incorporating verification matrix, algorithmic engines, and prototype validation.

[EXECUTIVE SUMMARY] *MSME360 is an integrated, secure, multi-tenant web application designed to resolve the pervasive operational fragmentation faced by Micro, Small, and Medium Enterprises (MSMEs). By unifying inventory control, sales order processing, procurement, double-entry financial ledger accounting, workforce administration, and real-time business intelligence into a cohesive cloud platform, MSME360 enforces ACID transactional integrity across distributed operational silos while eliminating expensive third-party licensing overheads.*

Table of Contents

1. Introduction **5**
1.1 Purpose and Intended Audience 5
1.2 System Scope and Objectives 5
1.3 Project Boundaries and Explicit Scope Exclusions 5
1.4 Definitions, Acronyms, and Abbreviations 6
1.5 Standards and Project References 7
1.6 Document Organization 7
2. Overall Description **8**
2.1 Product Perspective and Architectural Spectrum 8
2.2 High-Level Product Functions 9
2.3 User Classes and Operational Roles (RBAC) 9
2.4 Operating Environment and Target Infrastructure 11
2.5 Design, Implementation, and Academic Constraints 11
2.6 User Documentation Deliverables 12
2.7 Technical Assumptions and Operational Dependencies 12
3. System Features and Functional Requirements **13**
3.1 Module 1: Authentication, Authorization & User Management (FR-001 to FR-006) 13
3.2 Module 2: Product Catalog & Category Taxonomy (FR-007 to FR-012) 14
3.3 Module 3: Inventory Control & Stock Movement Audit Ledger (FR-013 to FR-018) 15
3.4 Module 4: Supplier Procurement & Purchase Order Lifecycle (FR-019 to FR-024) 16
3.5 Module 5: Sales Order Processing & Point-of-Sale (POS) Engine (FR-025 to FR-031) 17
3.6 Module 6: Customer Relationship Management (CRM) (FR-032 to FR-036) 18
3.7 Module 7: Financial Accounting & Cash Flow Tracking (FR-037 to FR-042) 19
3.8 Module 8: Workforce & Human Resources Administration (FR-043 to FR-046) 19
3.9 Module 9: Business Intelligence, Algorithmic Engine & Alerts (FR-047 to FR-052) 20
4. External Interface Requirements **22**
4.1 User Interfaces & Viewport Ergonomics 22
4.2 Hardware Interfaces 22
4.3 Software Interfaces & Data Exchange 23
4.4 Communication Interfaces & Protocols 23
5. Non-Functional Requirements (System Quality Attributes) **23**
5.1 Performance, Latency, and Throughput 23
5.2 Security, Cryptography, and Access Governance 24
5.3 Data Privacy and Confidentiality 24
5.4 Reliability, Fault Tolerance, and ACID Guarantees 24
5.5 Usability, Human Factors, and Accessibility 25
5.6 Maintainability, Modularity, and Code Health 25
5.7 Portability and Cross-Platform Compatibility 25
5.8 Scalability and Resource Utilization 25
5.9 Diagnostics, Auditing, and Error Handling 26
6. Data Requirements and Schema Architecture **27**
6.1 Logical Data Model & Entity Relationship Architecture 27
6.2 Data Dictionaries and Schema Specifications 27
6.3 Data Validation Rules and Integrity Constraints 29
6.4 Storage, Retention, and Archival Guidelines 30
6.5 Data Import and Export Formats 30
7. Operational Use Cases and System Workflows **30**
7.1 Use Case Specifications (UC-001 through UC-008) 30
8. System Constraints, Assumptions, and Limitations **36**

8.1 Mandatory Technical & Architectural Constraints 36
8.2 Validated Engineering Assumptions 36
8.3 Known Implementation Limitations of the Current Baseline 36
9. Verification Strategy and Acceptance Criteria **37**
9.1 Verification Methods and Quality Gates 37
9.2 Acceptance Criteria Matrix (AC-001 to AC-015) 37
10. Requirements Traceability Matrix (RTM) **38**
11. Appendices **39**
Appendix A: Comprehensive Technical Glossary 39
Appendix B: MoSCoW Prioritization Framework 40
Appendix C: Assumptions and Open Issues Register 40
Appendix D: Complete REST API Endpoint Inventory 41

1. Introduction

1.1 Purpose and Intended Audience

This Software Requirements Specification (SRS) establishes a complete, unambiguous, and verifiable specification of requirements for the MSME360 platform—an integrated web-based business management and analytics system tailored for Micro, Small, and Medium Enterprises (MSMEs). This document serves as the authoritative technical agreement among software engineering students, academic project evaluators, faculty supervisors, and future maintainers.

The primary intended audience comprises: (1) Senior Software Engineers and Technical Reviewers evaluating architectural consistency, data integrity protocols, and API contracts; (2) Quality Assurance (QA) and Test Engineers constructing verification suites, automated integration tests, and acceptance test plans; (3) Academic Project Guides and Evaluators reviewing compliance with academic milestones across Database Management Systems (DBMS), Design and Analysis of Algorithms (DAA), Software Engineering (SE), and Web Technology (WT); and (4) MSME Business Stakeholders seeking operational transparency into how core inventory, procurement, sales, and accounting workflows are centralized.

1.2 System Scope and Objectives

MSME360 provides a cloud-native, responsive Single Page Application (SPA) backed by high-performance asynchronous RESTful web services. The platform is designed to overcome the severe operational hurdles documented across the MSME sector—where enterprises collectively contribute over 30% of national GDP, 45% of manufacturing output, and employ 110+ million workers, yet remain crippled by fragmented software tools, manual ledger entry, and delayed visibility.

Core Objectives: The primary objectives of the MSME360 system encompass:

- **Unified Operations Management:** Consolidate inventory tracking, supplier purchasing, sales invoicing, customer CRM, general ledger accounting, and workforce management into a single unified cloud repository.
- **Granular Security Governance:** Enforce strict Role-Based Access Control (RBAC) protocols utilizing JSON Web Tokens (JWT) and modern password hashing to govern access across business owners, store managers, and front-desk staff.
- **Transactional Data Integrity:** Execute dual-sided atomic database transactions ensuring zero drift between physical stock counts, stock movement audit trails, and revenue/expense ledger entries.
- **Actionable Business Intelligence:** Deliver dynamic operational intelligence through automated low-stock reorder point algorithms, stock velocity categorization, debtor aging buckets, and real-time interactive business dashboards.

1.3 Project Boundaries and Explicit Scope Exclusions

To maintain strict engineering feasibility within the academic timeline while ensuring industrial-grade implementation quality, the system boundaries for the current project release (Version 1.0) are explicitly defined. Capabilities not scheduled for delivery are cataloged below to prevent scope ambiguity.

> **[EXPLICIT SCOPE BOUNDARY]** *The current scope release of MSME360 focuses exclusively on internal operational management, inventory control, billing ledger recording, and business analytics. Direct financial settlements through commercial payment gateways (e.g., live Razorpay/Stripe webhooks) and automated submissions to government taxation portals (e.g., GSTN e-filing, live e-Way Bill generation) are strictly outside the Version 1.0 project boundary.*

> *All sales and purchases generate internal accounting ledgers and receipts, but external payment settlement is treated as external manual cash, bank transfer, or offline reconciliation.*

Explicit Scope Exclusions (Version 1.0): The following modules and integrations are explicitly excluded from the current baseline:

- **Live Banking & Payment Gateway APIs:** No live settlement transactions with external acquiring banks or card networks are executed within the application.
- **Direct Government Tax Portal Integration:** Tax rates (GST/VAT) are computed and tracked within internal invoices and reports, but automated cryptographic signing and transmission to taxation authorities are deferred.
- **Automated Statutory Payroll Withholding:** Basic employee compensation records and departmental salaries are tracked, but complex multi-tier statutory withholdings (PF, ESI, professional tax slabs) are not automated.
- **Offline-First Progressive Web App (PWA):** The web application requires active network connectivity to synchronize with the centralized REST API; offline caching with background service workers is reserved for future versions.
- **Deep Learning Demand Forecasting:** Dynamic Reorder Points and Stock Velocity are computed deterministically; deep neural network predictive forecasting using external macroeconomic regressors is outside the current scope.

1.4 Definitions, Acronyms, and Abbreviations

Table 1.1 defines technical terms, architectural acronyms, and domain-specific terminology utilized throughout this document.

Acronym / Term	Formal Definition and Context within MSME360
MSME	Micro, Small, and Medium Enterprises. The primary target demographic, characterized by resource constraints, lean operational staff, and high sensitivity to enterprise software licensing costs.
SRS	Software Requirements Specification. Formal document describing the functional, performance, and quality requirements of the system.
RBAC	Role-Based Access Control. Security governance model restricting API endpoint execution and UI views based on assigned enterprise roles (Admin, Owner, Manager, Staff).
ACID	Atomicity, Consistency, Isolation, Durability. Transactional properties enforced during stock movements and accounting updates to prevent partial state corruption.
JWT	JSON Web Token (RFC 7519). Compact, URL-safe means of representing claims between two parties, used for stateless authentication.
SKU	Stock Keeping Unit. Unique alphanumeric code assigned to each distinct sellable product variant for inventory tracking and barcode association.
ROP	Dynamic Reorder Point. Algorithmic stock threshold triggering procurement replenishment based on average daily usage, supplier lead time, and safety stock.

Acronym / Term	Formal Definition and Context within MSME360
GRN	Goods Received Note. Formal inventory record validating physical receipt of items against an issued supplier Purchase Order.
3NF	Third Normal Form. Relational database normalization standard eliminating transitive dependencies to maintain high update anomalies resistance.
ORM	Object-Relational Mapping. Database abstraction layer (SQLAlchemy 2.0) mapping relational tables to typed Python domain entities.
REST	Representational State Transfer. Stateless architectural style utilizing HTTP verbs (GET, POST, PUT, DELETE) for decoupled client-server data exchange.
SPA	Single Page Application. Modern web architecture (React 18) loading a single HTML shell and dynamically re-rendering views via client-side routing.
DAA	Design and Analysis of Algorithms. Core computer engineering subject covering computational complexity, optimization, and deterministic algorithms.
DBMS	Database Management Systems. Core computer engineering subject covering relational data modeling, indexing, query optimization, and concurrency control.
Debtor Aging	Accounting technique bucketing unpaid customer accounts receivable into time tranches (0–30, 31–60, 61–90, 90+ days) to manage default risk.

1.5 Standards and Project References

This specification adheres to established requirements engineering standards and cites verified technical references:

- **IEEE Std 830-1998:** Recommended Practice for Software Requirements Specifications (organizational structure and requirement taxonomy).
- **ISO/IEC/IEEE 29148:2018:** Systems and software engineering — Life cycle processes — Requirements engineering (verifiability, atomicity, and necessity guidelines).
- **MSME360 Project Synopsis:** Group No. 5, Academic Year 2026–27, Computer Engineering Department, Faculty Guide: Prof. Jignesh Sisodia.
- **MSME360 Architectural Presentation:** Comprehensive 10-slide architectural overview, data models, algorithm formulations, and DevOps pipelines.
- **MSME360 Source Code Repository:** Verified implementation containing Python FastAPI backend, Pydantic schemas, React 18 frontend, and Pytest verification suite.
- **Reference Mini-Project SRS:** Departmental benchmark document: Financial Fraud Detection System (Vandan Jethwa, Soham Gurav, Kushal Karkal; Guide: Prof. Reeta Koshy).
- **Technical Framework Documentation:** FastAPI Framework Documentation (v0.110+), React 18 Core Documentation, PostgreSQL 16 Reference Manual, and SQLAlchemy 2.0 Guide.

1.6 Document Organization

This SRS is organized into eleven formal sections in accordance with industrial documentation best practices. Section 1 outlines purpose, scope, definitions, and references. Section 2 establishes product perspective,

operational functions, user classifications, operating environments, and design constraints. Section 3 details atomic functional requirements organized across nine operational modules. Section 4 specifies external interfaces (UI, hardware, software, communications). Section 5 establishes testable non-functional quality attributes. Section 6 defines the 3NF data architecture and data dictionaries. Section 7 presents formal operational use cases. Section 8 details technical constraints and limitations. Section 9 outlines verification and acceptance criteria. Section 10 provides a complete bidirectional Requirements Traceability Matrix (RTM). Section 11 contains supplementary appendices including glossary, MoSCoW criteria, open issues, and API catalogs.

2. Overall Description

2.1 Product Perspective and Architectural Spectrum

MSME operations are historically plagued by a stark technological divide. At the top end, Enterprise Resource Planning (ERP) systems (e.g., SAP S/4HANA, NetSuite) offer comprehensive operational coverage but impose prohibitive licensing expenditures, multi-month deployment cycles, and steep learning curves that alienate lean small business teams. Conversely, modular SaaS suites (e.g., Zoho, Odoo) introduce escalating recurring subscription costs and cross-module synchronization latency. Consequently, the vast majority of MSMEs revert to isolated desktop billing utilities (e.g., Tally Prime, Busy) or manual spreadsheets (Microsoft Excel, Google Sheets). These desktop tools operate as disconnected data silos lacking native cloud multi-user collaboration, while spreadsheets are notoriously prone to silent data corruption, lack audit trails, and provide zero referential integrity.

Table 2.1 highlights the comparative research gap motivating the development of the MSME360 platform.

Solution Class	Representative Systems	Operational Strengths	Critical Weaknesses & Research Gap
Enterprise ERPs	SAP, NetSuite, Dynamics 365	Extremely robust, comprehensive functional coverage, enterprise auditability.	Prohibitive licensing costs, massive operational overhead, multi-month deployment.
SaaS Modular Suites	Zoho One, Odoo Enterprise	Modular, cloud-native, scalable across business growth phases.	High recurring per-user fees, cross-app data latency, integration complexity.
Desktop Billing Tools	Tally Prime, Busy Accounting	Familiar accounting interface, high local adoption, robust offline billing.	Isolated data silos, no native cloud multi-user access, manual data consolidation.
Manual Spreadsheets	Microsoft Excel, Google Sheets	Zero software acquisition cost, highly customizable ad-hoc layouts.	Error-prone, lack concurrent access controls, zero referential integrity, non-scalable.
MSME360 (Proposed)	Integrated Web-Based Business Platform	Lightweight, centralized 3NF cloud database, native RBAC, atomic transactions, real-time BI.	Explicitly targeted at MSME operational scale; excludes complex multi-national taxation.

System Architecture Perspective: Architecturally, MSME360 is engineered as a modern, decoupled three-tier web application:

- **Presentation Tier (Client):** Implemented as a responsive Single Page Application (SPA) utilizing React 18, Vite, TypeScript, and Tailwind CSS. The presentation layer handles local input validation, dynamic component rendering, and asynchronous HTTP communication via Axios/Fetch.
- **Application & Security Tier (Server):** Powered by Python 3.12 and the FastAPI asynchronous framework. It implements RESTful API routing, strict request/response data validation via Pydantic models, role-based dependency injection guards, and atomic business orchestration.
- **Data & Analytics Tier (Storage):** Anchored by a PostgreSQL relational database operating in Third Normal Form (3NF) with foreign key cascading rules and composite B-tree indexes. Database migrations are version-controlled via Alembic, while object persistence is mediated through SQLAlchemy 2.0 ORM. Real-time analytics are processed through dedicated analytical query aggregators.

2.2 High-Level Product Functions

The MSME360 platform consolidates eight core operational subsystems into a cohesive operational workflow:

- **1. Authentication & Security:** Centralized JWT-based authentication with bcrypt password encryption and role-gated route middleware.
- **2. Product Catalog Management:** Hierarchical category classification, product cataloging, SKU assignment, and dual-pricing (cost vs. selling).
- **3. Inventory & Stock Control:** Real-time stock level monitoring, threshold alerts, and an immutable audit ledger of all movements (Purchases, Sales, Adjustments, Returns).
- **4. Procurement & Purchases:** Supplier tracking, multi-item purchase order generation, automated unit cost recalculation, and stock replenishment.
- **5. Sales & POS Engine:** Customer checkout terminal, multi-item invoicing, tax and discount computations, and immediate atomic stock decrements.
- **6. Customer Relationship Management:** Customer profiles, contact registries, purchase histories, and cumulative lifetime spending analytics.
- **7. Finance & Accounting:** Automated income/expense transaction logging, gross and net profit tracking, profit margin calculations, and receivables/payables tracking.
- **8. Workforce Administration:** Employee roster maintenance, departmental assignments, salary records, and internal role provisioning.
- **9. BI & Reporting Engine:** Dynamic executive dashboards, sales velocity trend charts, automated stock alert evaluations, and structured CSV/JSON report exports.

2.3 User Classes and Operational Roles (RBAC)

MSME360 enforces strict segregation of duties through a four-tier Role-Based Access Control (RBAC) hierarchy. Table 2.2 defines the operational user classes, their system responsibilities, and access boundaries.

User Role	Operational Responsibilities	System Access Boundaries	Technical Competence

User Role	Operational Responsibilities	System Access Boundaries	Technical Competence
System Administrator (ADMIN)	System configuration, user provisioning, role assignments, global database maintenance, and technical audit logging.	Unrestricted access across all operational endpoints, administrative routes, and system logs.	Advanced technical proficiency in IT administration, user security, and database concepts.
Business Owner (OWNER)	Strategic business oversight, executive financial monitoring, profit margin review, payroll approvals, and business report generation.	Full operational access to Finance, Sales, Purchases, Inventory, HR, and Analytics. Cannot alter system code or low-level configurations.	Business domain literacy; comfortable with executive metric dashboards, profit/loss statements, and reports.
Store Manager (MANAGER)	Day-to-day warehouse operations, purchase order execution, stock adjustments, supplier communication, and customer dispute resolution.	Read/write access to Inventory, Purchases, Sales, and Customers. Read-only access to staff directory. No access to financial margins or net profit ledgers.	Operational proficiency in inventory replenishment, retail billing, and stock taking.
Sales Staff (STAFF)	Point-of-sale customer checkout, receipt generation, customer registration, and basic stock availability inquiries.	Restricted to creating sales transactions, adding new customer records, and querying available product stock. No access to purchase costs, margins, or HR.	Basic computer literacy; capability to operate a modern web browser and input sales items.

Table 2.3 defines the authoritative Role-Permission Matrix across all major functional modules:

Functional Module / Operational Capability	ADMIN	OWNER	MANAGER	STAFF
User Provisioning & Role Modification	Full Access	Create / View	No Access	No Access
Product Catalog & Category Creation / Editing	Full Access	Full Access	Full Access	Read-Only
Stock Level Queries & Availability Lookup	Full Access	Full Access	Full Access	Read-Only
Manual Stock Adjustments (Audit Reconciliations)	Full Access	Full Access	Full Access	No Access
Supplier Management & Purchase Order Creation	Full Access	Full Access	Full Access	No Access
Purchase Order Cancellation & Reversal	Full Access	Full Access	No Access	No Access
Customer Registration & Profile Updates	Full Access	Full Access	Full Access	Create / View

Functional Module / Operational Capability	ADMIN	OWNER	MANAGER	STAFF
Point-of-Sale Checkout & Sales Receipt Issuance	Full Access	Full Access	Full Access	Full Access
Sales Order Cancellation & Return Processing	Full Access	Full Access	Full Access	No Access
General Financial Summary (Revenues, Profits, Margins)	Full Access	Full Access	No Access	No Access
Detailed Financial Transactions Ledger	Full Access	Full Access	No Access	No Access
Employee Profile Creation & Salary Management	Full Access	Full Access	Read-Only	No Access
Executive Dashboard & Daily Sales KPI Cards	Full Access	Full Access	Operational Only	Restricted
Business Intelligence Reports & CSV/JSON Export	Full Access	Full Access	Sales / Stock Only	No Access

2.4 Operating Environment and Target Infrastructure

MSME360 is engineered to operate across modern distributed cloud infrastructure while maintaining a lightweight footprint capable of local workstation execution during academic demonstration and testing.

- **Application Server:** Python 3.12+ 64-bit runtime environment running the Uvicorn ASGI server. Containerized via multi-stage Docker images to guarantee reproducible execution across Linux (Ubuntu 22.04 LTS / Debian 12) and Windows 11 environments.
- **Database Engine:** PostgreSQL 15+ relational database with standard connection pooling. For rapid local testing and CI/CD automation, the architecture supports an in-memory repository implementation adhering strictly to identical interface contracts.
- **Client Web Browser:** Compiled modern JavaScript (ES2022) bundle executed in modern evergreen web browsers: Google Chrome (v110+), Mozilla Firefox (v110+), Microsoft Edge (v110+), and Apple Safari (v16+). Minimum supported viewport: 1024x768 pixels.
- **Production Cloud Hosting:** Containerized backend deployment targeting Render Cloud Web Services; frontend static SPA hosted via Vercel Edge Network with SSL termination.

2.5 Design, Implementation, and Academic Constraints

The architecture and implementation of MSME360 are governed by rigorous engineering and academic constraints:

- **Architectural Decoupling:** The backend must strictly separate presentation, routing, validation, business orchestration, and persistence across decoupled layers.
- **Academic Syllabus Alignment:** The system must map directly to core subjects in the Computer Engineering curriculum: (1) DBMS (3NF schema, cascading constraints, B-tree indexing, ACID transactions); (2) DAA (Dynamic Reorder Point algorithms, Stock Velocity sorting, Debtor Aging analysis); (3) Software Engineering (Agile incremental delivery, traceability, verification matrices); and (4) Web Technology (FastAPI REST APIs, React 18 SPA, responsive Tailwind CSS).

> **[ARCHITECTURAL RESOLUTION: BACKEND STACK CONFLICT]** *An inspection of the workspace project slide deck*

> *('SaaS Business Suites.pdf') identified a minor slide template artifact on Slide 9 mentioning 'Spring Boot REST APIs' and 'Spring Data IPA'. In accordance with the project source hierarchy rules, this has been reconciled against the authoritative Project Proposal Document ('G5_2024300118,2024300129,2024300133.pdf') and the verified, fully functioning codebase. The binding technology stack is definitively established as Python 3.12, FastAPI, Pydantic, SQLAlchemy 2.0, PostgreSQL, React 18, and Vite. The legacy slide template text is formally cataloged in Appendix C (Open Issues Register).*

2.6 User Documentation Deliverables

The project submission includes comprehensive documentation deliverables:

- **1. System Administrator & Deployment Guide:** Comprehensive guide detailing environment configuration, Docker container initialization, PostgreSQL connection strings, and Alembic database migrations.
- **2. End-User Operational Manual:** Role-specific operational instructions detailing catalog creation, point-of-sale checkout, stock taking, and financial analytics.
- **3. Interactive OpenAPI / Swagger Documentation:** Auto-generated, interactive API specification accessible at `/docs` (Swagger UI) and `/redoc` detailing all request schemas, headers, status codes, and error models.
- **4. Postman Integration Test Collection:** Pre-configured Postman workspace containing automated integration test scripts covering authentication, inventory, sales, purchases, and reporting.

2.7 Technical Assumptions and Operational Dependencies

The functional validity of the requirements specified herein relies on the following engineering assumptions:

- **Network Reliability:** Client workstations maintain an active broadband internet or local intranet connection with latency < 250ms to the backend API host.
- **Master Data Accuracy:** Initial product master data, historical stock balances, and opening customer records are loaded via valid CSV files or administrator setup before live transactions commence.
- **Operational Integrity:** Users adhere strictly to credential confidentiality protocols and do not share administrative JWT bearer tokens.
- **Temporal Consistency:** Underlying operating system environments provide an accurate Network Time Protocol (NTP) synchronized UTC clock for transaction timestamps.

3. System Features and Functional Requirements

This section details the functional requirements of the MSME360 platform, partitioned across nine cohesive operational modules. Each module is documented with its operational context, participating actors, preconditions, triggers, step-by-step main workflows, alternative and exception flows, postconditions, and atomic requirement statements adhering to the ISO/IEC/IEEE 29148 standard. Requirement statements utilize the mandatory normative keyword 'shall' and are assigned stable, traceable identifiers (FR-001 through FR-052) alongside MoSCoW priorities.

3.1 Module 1: Authentication, Authorization & User Management

Module 1 governs identity verification, cryptographic session management, and role-based operational gating across all system functions.

- **Participating Actors:** System Administrator (ADMIN), Business Owner (OWNER), Store Manager (MANAGER), Sales Staff (STAFF).
- **Preconditions:** User account exists in persistent storage with an active status flag (`is_active = True`) and a bcrypt-hashed password.
- **Trigger:** User submits authentication credentials (username and plaintext password) via the web login interface.
- **Main Success Scenario:** 1. Client POSTs credentials to `/api/auth/login`. 2. System retrieves user record by username. 3. System verifies plaintext password against stored cryptographic hash using `passlib.pwd_context` (bcrypt). 4. System generates a signed JWT bearer token containing `sub` (username), `role`, and expiration timestamp (24 hours). 5. System returns token and serialized user profile. 6. Client stores token and includes it in subsequent HTTP `Authorization: Bearer <token>` headers.
- **Alternative Flows:** A1: Active token exists on client startup; client sends GET `/api/auth/me` to refresh user context without prompting for password.
- **Exception Flows:** E1: Username not found or password verification fails → System returns HTTP 401 Unauthorized with error code `INVALID_CREDENTIALS`. E2: User account is deactivated (`is_active = False`) → System returns HTTP 403 Forbidden. E3: Expired or cryptographically tampered token presented → System returns HTTP 401 Unauthorized.
- **Postconditions:** Authenticated session is established; API gateway enforces role permissions based on decoded token claims.

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-001	The system shall authenticate users via username and password, issuing a signed JSON Web Token (JWT) using the HS256 algorithm upon successful credential verification.	Must	Automated API Test
FR-002	The system shall store all user passwords exclusively in hashed form utilizing the bcrypt cryptographic hashing algorithm with a minimum work factor of 12.	Must	Code Inspection / Test
FR-003	The system shall enforce an expiration duration of exactly 24 hours on all issued JWT access tokens, after which re-authentication is mandatory.	Must	Automated Security Test

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-004	The system shall provide an endpoint `/api/auth/me` allowing authenticated clients to retrieve their current user profile and assigned role.	Must	Automated API Test
FR-005	The system shall enforce Role-Based Access Control (RBAC) across all protected API routes, returning HTTP 403 Forbidden whenever an authenticated user lacks the necessary role permissions.	Must	Automated Security Test
FR-006	The system shall allow users with the ADMIN or OWNER role to provision new user accounts with designated roles (ADMIN, OWNER, MANAGER, STAFF).	Must	Automated Functional Test

3.2 Module 2: Product Catalog & Category Taxonomy

Module 2 manages master product definitions, hierarchical category classifications, unit pricing, and stock monitoring thresholds.

- **Participating Actors:** ADMIN, OWNER, MANAGER (Write/Edit); STAFF (Read-only).
- **Preconditions:** Target category must exist prior to associating a product. User must possess MANAGER, OWNER, or ADMIN role.
- **Trigger:** User navigates to Inventory page and initiates category or product creation.
- **Main Success Scenario:** 1. User enters product name, unique SKU, category ID, cost price, selling price, opening stock, and minimum stock threshold. 2. Client submits POST `/api/inventory/products`. 3. System validates SKU uniqueness, positive pricing constraints, and category existence. 4. System persists product record and initializes stock balance. 5. System logs initial movement audit record. 6. System returns HTTP 201 Created with complete product payload.
- **Alternative Flows:** A1: Product updated via PUT `/api/inventory/products/{id}` with modified prices or reorder thresholds.
- **Exception Flows:** E1: Duplicate SKU submitted → System returns HTTP 400 Bad Request with code `DUPLICATE_SKU`. E2: Non-positive price submitted (`cost_price <= 0` or `selling_price <= 0`) → System returns HTTP 422 Unprocessable Entity. E3: Non-existent category ID supplied → System returns HTTP 404 Not Found.
- **Postconditions:** New product is active in the catalog and available for procurement and sales transactions.

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-007	The system shall allow authorized users (ADMIN, OWNER, MANAGER) to create and maintain product categories with a unique category name and optional description.	Must	Automated Functional Test
FR-008	The system shall enforce global uniqueness on all Stock Keeping Unit (SKU) codes across the product catalog.	Must	Database / API Test
FR-009	The system shall validate that both cost_price and selling_price are strictly greater than zero upon product creation and modification.	Must	Schema Validation Test

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-010	The system shall support updating product attributes including name, category, pricing, and minimum stock threshold via PUT `/api/inventory/products/{id}`.	Must	Automated Functional Test
FR-011	The system shall allow users across all authenticated roles to list and search products by name, SKU, or category with real-time stock availability.	Must	Automated Functional Test
FR-012	The system shall dynamically assign a stock_status indicator ('IN_STOCK', 'LOW_STOCK', 'OUT_OF_STOCK') to each product based on current quantity relative to min_stock_level.	Must	Automated Functional Test

3.3 Module 3: Inventory Control & Stock Movement Audit Ledger

Module 3 maintains real-time physical inventory balances, executes manual stock reconciliations, and records an immutable historical audit trail of all quantity variations.

- **Participating Actors:** ADMIN, OWNER, MANAGER.
- **Preconditions:** Target product exists and is active. User is authorized for stock adjustments.
- **Trigger:** Physical stock discrepancy discovered during warehouse audit or damaged goods write-off.
- **Main Success Scenario:** 1. User submits product ID, quantity change (positive or negative integer), movement type (`ADJUSTMENT`, `RETURN`), and reason string. 2. System validates that resulting stock balance will not drop below zero. 3. System updates `stock_quantity`. 4. System records an immutable `StockMovement` entry capturing previous stock, new stock, timestamp, movement type, and reason. 5. System re-evaluates stock threshold status.
- **Alternative Flows:** A1: Stock increment resulting from customer return or inventory recount.
- **Exception Flows:** E1: Negative adjustment exceeds available stock → System returns HTTP 400 Bad Request with code `INSUFFICIENT_STOCK`. E2: Unrecognized movement type → System returns HTTP 422 Unprocessable Entity.
- **Postconditions:** Stock balance is updated and full audit record is appended to the movement ledger.

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-013	The system shall automatically record an immutable stock movement audit record for every inventory change, documenting product_id, movement_type, quantity, previous_stock, new_stock, reason, and UTC timestamp.	Must	Integration Test
FR-014	The system shall support four distinct stock movement classifications: PURCHASE, SALE, ADJUSTMENT, and RETURN.	Must	Automated Schema Test
FR-015	The system shall allow authorized users (ADMIN, OWNER, MANAGER) to perform manual stock adjustments with mandatory audit reason capture.	Must	Automated Functional Test
FR-016	The system shall reject any inventory adjustment or decrement that would result in a negative stock balance (`stock_quantity < 0`).	Must	Boundary Value Test

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-017	The system shall provide an endpoint `/api/inventory/movements` to retrieve the complete chronological history of stock movements with filtering capabilities.	Must	Automated API Test
FR-018	The system shall automatically re-evaluate product stock status and trigger system alert notifications whenever stock drops to or below min_stock_level.	Must	Event / Integration Test

3.4 Module 4: Supplier Procurement & Purchase Order Lifecycle

Module 4 manages supplier interactions, inbound purchase orders, stock replenishment, and automatic expense ledger recording.

- **Participating Actors:** ADMIN, OWNER, MANAGER.
- **Preconditions:** Catalog products exist. Supplier details are known. User is authorized for procurement.
- **Trigger:** Procurement officer submits a completed purchase order receipt.
- **Main Success Scenario:** 1. User submits supplier name, line items (product IDs, quantities, unit prices), tax rate, and discount. 2. System computes item subtotals, order subtotal, tax amount, and net purchase total. 3. System persists purchase record with `status = COMPLETED`. 4. System automatically increments `stock_quantity` for each product. 5. System logs `PURCHASE` movements in the audit ledger. 6. System creates an `EXPENSE` financial transaction linked to the purchase ID.
- **Alternative Flows:** A1: Order cancelled via POST `/api/business/purchases/{id}/cancel` by ADMIN/OWNER; system decrements previously added stock and marks order `CANCELLED`.
- **Exception Flows:** E1: Empty item list submitted → System returns HTTP 400 with code `EMPTY_ITEMS`. E2: Invalid product ID in line items → System returns HTTP 404. E3: Cancellation requested on already cancelled order → System returns HTTP 400 `ALREADY_CANCELLED`.
- **Postconditions:** Inventory is replenished and total purchase cost is recorded in accounts payable / expenses.

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-019	The system shall allow authorized users (ADMIN, OWNER, MANAGER) to create purchase orders specifying supplier name, itemized product quantities, unit costs, tax rate, and discount.	Must	Automated Functional Test
FR-020	The system shall validate that a purchase order contains at least one valid product line item with quantity strictly greater than zero.	Must	Schema Validation Test
FR-021	The system shall automatically increment inventory balances for all line-item products upon purchase order completion within an atomic transaction.	Must	Integration / DB Test
FR-022	The system shall automatically append an EXPENSE financial transaction to the general ledger upon purchase order creation, capturing supplier name, amount, and reference ID.	Must	Integration Test

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-023	The system shall allow users with the ADMIN or OWNER role to cancel an active purchase order, reversing stock increments and updating order status to CANCELLED.	Should	Automated Functional Test
FR-024	The system shall provide an endpoint `/api/business/purchases` returning all purchase orders with supplier details, line items, timestamps, and payment statuses.	Must	Automated API Test

3.5 Module 5: Sales Order Processing & Point-of-Sale (POS) Engine

Module 5 executes customer billing, point-of-sale transactions, atomic inventory decrements, and automated revenue ledger postings.

- **Participating Actors:** ADMIN, OWNER, MANAGER, STAFF.
- **Preconditions:** Customer profile selected or guest mode activated. Products are active and possess sufficient stock.
- **Trigger:** Sales operator clicks 'Complete Sale' at the sales checkout terminal.
- **Main Success Scenario:** 1. User submits customer ID (optional), line items (product IDs, quantities, optional price overrides), tax rate, and discount. 2. System verifies that available stock meets or exceeds requested quantity for every item. 3. System calculates line subtotals, aggregate tax, net discount, and total payable amount. 4. System persists sale order record. 5. System atomically decrements stock for each product and records `SALE` audit entries. 6. System creates an `INCOME` financial transaction. 7. If registered customer, system increments customer's `total_spent`. 8. System returns completed sale invoice.
- **Alternative Flows:** A1: Guest sale executed without customer ID; invoice displays 'Guest Customer'. A2: Sale cancelled via POST `/api/business/sales/{id}/cancel`; stock is restored via `RETURN` movement and customer spending is decremented.
- **Exception Flows:** E1: Insufficient stock for any requested product → System aborts transaction and returns HTTP 400 Bad Request with code `INSUFFICIENT_STOCK` detailing available vs. requested units. E2: Negative discount or invalid tax rate → System returns HTTP 422.
- **Postconditions:** Sale invoice generated, physical stock decremented atomically, revenue posted to ledger.

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-025	The system shall allow users across all roles (ADMIN, OWNER, MANAGER, STAFF) to create sales transactions specifying customer, line items, tax rate, and discount.	Must	Automated Functional Test
FR-026	The system shall verify stock availability for every requested line item prior to finalizing a sale, rejecting the transaction if available stock is insufficient.	Must	Automated Boundary Test
FR-027	The system shall atomically decrement product stock quantities and generate corresponding SALE movement records upon sale completion.	Must	Integration / ACID Test

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-028	The system shall automatically append an INCOME financial transaction to the ledger upon sale completion, recording total revenue and sale reference ID.	Must	Integration Test
FR-029	The system shall automatically update the customer's cumulative total_spent attribute whenever a sale associated with a valid customer ID is completed.	Must	Integration Test
FR-030	The system shall allow authorized users (ADMIN, OWNER, MANAGER) to cancel a sale, returning items to stock via RETURN movements and reversing customer spend.	Should	Automated Functional Test
FR-031	The system shall provide endpoints `/api/business/sales` and `/api/business/sales/{id}` to list and retrieve itemized sales invoices.	Must	Automated API Test

3.6 Module 6: Customer Relationship Management (CRM)

Module 6 maintains customer master records, tracks purchase frequency, and monitors customer lifetime spend.

- **Participating Actors:** ADMIN, OWNER, MANAGER, STAFF.
- **Preconditions:** User possesses valid authenticated session.
- **Trigger:** New customer onboarded at checkout or through customer management interface.
- **Main Success Scenario:** 1. User submits customer name, optional email, and phone number. 2. System validates contact formatting. 3. System creates customer profile with `total_spent = 0.0` and `is_active = True`. 4. System returns HTTP 201 Created.
- **Alternative Flows:** A1: Customer searched by name or email substring via `/api/business/customers?search={query}`.
- **Exception Flows:** E1: Invalid email format supplied → System returns HTTP 422 Unprocessable Entity. E2: Customer ID not found during update → System returns HTTP 404.
- **Postconditions:** Customer record active and selectable in sales workflow.

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-032	The system shall allow users across all roles to register customer records capturing name, optional email, and optional phone number.	Must	Automated Functional Test
FR-033	The system shall support updating existing customer contact information via PUT `/api/business/customers/{id}` for authorized roles (ADMIN, OWNER, MANAGER).	Must	Automated Functional Test
FR-034	The system shall support searching and filtering customer records in real time using name or email substring matching.	Must	Automated API Test
FR-035	The system shall track and display cumulative total_spent across all completed sales transactions for each customer profile.	Must	Integration Test
FR-036	The system shall support soft deactivation of customer records to preserve historical sales referential integrity.	Should	Database Integrity Test

3.7 Module 7: Financial Accounting & Cash Flow Tracking

Module 7 consolidates revenues, procurement expenses, operational overheads, and computes core financial metrics including gross profit, net profit, and profit margins.

- **Participating Actors:** ADMIN, OWNER.
- **Preconditions:** User authenticated with ADMIN or OWNER role. Financial ledger populated with sales and expense entries.
- **Trigger:** Business owner requests financial health summary via GET `/api/finance/summary`.
- **Main Success Scenario:** 1. System aggregates all `INCOME` transactions and completed sales revenues. 2. System aggregates all `EXPENSE` transactions and completed purchase expenditures. 3. System calculates `gross_profit = total_revenue - purchases_expense`. 4. System calculates `net_profit = total_revenue - total_expenses`. 5. System calculates `profit_margin_percent = (net_profit / total_revenue) * 100`. 6. System compiles total receivables and payables. 7. System returns JSON payload.
- **Alternative Flows:** A1: User queries detailed transaction log via `/api/finance/transactions`.
- **Exception Flows:** E1: User with MANAGER or STAFF role attempts to access financial summary → System returns HTTP 403 Forbidden.
- **Postconditions:** Financial summary displayed on Executive Dashboard.

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-037	The system shall restrict access to general ledger summaries, net profit calculations, and financial transactions exclusively to the ADMIN and OWNER roles.	Must	Security / Role Test
FR-038	The system shall automatically compute total revenue by aggregating all manual income transactions and completed sales totals.	Must	Automated Financial Test
FR-039	The system shall automatically compute total expenses by aggregating all operational expense entries and completed purchase totals.	Must	Automated Financial Test
FR-040	The system shall compute gross profit, net profit, and net profit margin percentage in real time based on active ledger records.	Must	Calculation Precision Test
FR-041	The system shall provide a chronologically ordered financial transaction log detailing transaction ID, type (INCOME/EXPENSE), category, amount, description, and timestamp.	Must	Automated API Test
FR-042	The system shall track outstanding accounts receivable and accounts payable balances for financial liquidity monitoring.	Should	Automated Financial Test

3.8 Module 8: Workforce & Human Resources Administration

Module 8 manages employee profiles, organizational departments, operational roles, and compensation baselines.

- **Participating Actors:** ADMIN, OWNER (Create/Edit); MANAGER (Read-only).
- **Preconditions:** User authenticated with ADMIN or OWNER role.
- **Trigger:** HR manager creates new employee record.

- **Main Success Scenario:** 1. User submits employee name, corporate email, operational role, department, and monthly base salary. 2. System validates that salary > 0. 3. System persists employee record. 4. System returns HTTP 201 Created.
- **Alternative Flows:** A1: Departmental roster queried via GET `/api/business/employees`.
- **Exception Flows:** E1: Non-positive salary submitted → System returns HTTP 422. E2: Unauthorized role attempts employee creation → System returns HTTP 403 Forbidden.
- **Postconditions:** Employee record is active in the enterprise directory.

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-043	The system shall allow users with the ADMIN or OWNER role to create employee records capturing name, email, role, department, and salary.	Must	Automated Functional Test
FR-044	The system shall validate that employee salary values are strictly positive numerical values (`salary > 0`).	Must	Schema Validation Test
FR-045	The system shall allow users with ADMIN, OWNER, or MANAGER roles to view the active employee directory.	Must	Security / Role Test
FR-046	The system shall support soft deactivation of employee records (`is_active = False`) upon employee termination.	Should	Automated Functional Test

3.9 Module 9: Business Intelligence, Algorithmic Engine & Reporting

Module 9 implements core analytical engines, deterministic inventory algorithms mapped to Design and Analysis of Algorithms (DAA), automated threshold alerts, and structured report exports.

- **Participating Actors:** ADMIN, OWNER, MANAGER.
- **Preconditions:** System populated with operational sales, inventory, and purchase data.
- **Trigger:** User accesses Executive Dashboard, triggers alert evaluation, or requests a sales report.
- **Main Success Scenario:** 1. System executes analytics aggregation service. 2. System applies algorithmic evaluations: Dynamic Reorder Point (ROP), Stock Velocity Classification, and Debtor Aging Analysis. 3. System compiles daily sales trends and stock turnover rates. 4. System evaluates active alerts. 5. System renders interactive visual charts (Recharts) or returns formatted CSV/JSON export.
- **Alternative Flows:** A1: User filters sales report by start date, end date, and category ID; system exports filtered dataset.
- **Exception Flows:** E1: Invalid date format in report filter request → System returns HTTP 422.
- **Postconditions:** Operational metrics and alerts presented to decision makers.

Core Algorithmic Formulations (DAA Alignment): The algorithmic engine executes three primary deterministic models mapped to DAA curriculum requirements:

- **1. Dynamic Reorder Point (ROP):** ROP = (Average Daily Usage * Lead Time) + Safety Stock. The system dynamically computes the replenishment trigger for each SKU based on 30-day historical sales velocity, supplier delivery lead time, and statistical buffer stock.
- **2. Stock Velocity Classification:** Categorizes inventory items into Fast-Moving, Medium-Moving, and Slow-Moving bands based on unit turnover rate, enabling targeted capital allocation.
- **3. Debtor Aging Analysis:** Segments outstanding receivables into aging tranches (0–30 days, 31–60 days, 61–90 days, 90+ days) to facilitate proactive debt collection.

Req ID	Functional Requirement Statement	Priority	Verification Method
FR-047	The system shall provide an endpoint `/api/dashboard/summary` returning daily sales, monthly revenue, monthly expenses, gross profit, profit margin, total products, and low-stock count.	Must	Automated API Test
FR-048	The system shall provide an endpoint `/api/analytics/sales` returning historical daily sales volume trends and top-selling products by quantity and revenue.	Must	Automated API Test
FR-049	The system shall provide an endpoint `/api/analytics/inventory` returning stock turnover rate, low-stock frequency, out-of-stock count, and category distribution.	Must	Automated API Test
FR-050	The system shall dynamically evaluate and generate system alerts for products with stock_quantity <= min_stock_level (LOW_STOCK) and stock_quantity <= 0 (OUT_OF_STOCK) with appropriate severity levels (WARNING, CRITICAL).	Must	Automated Logic Test
FR-051	The system shall provide an endpoint `/api/reports/sales` supporting filtered report generation by start_date, end_date, and category_id.	Must	Automated API Test
FR-052	The system shall support exporting generated business reports and database table views into structured CSV and JSON file formats.	Must	Export Verification Test

4. External Interface Requirements

4.1 User Interfaces & Viewport Ergonomics

The MSME360 user interface is delivered as a modern Single Page Application (SPA) styled with Tailwind CSS utility classes and Lucide React iconography. The UI adheres to an ergonomic dual-panel layout: a persistent collapsible vertical navigation sidebar on the left and a responsive content viewport on the right, capped by a top navigation bar.

Screen Hierarchy & Layout Specifications: The interface incorporates eight primary operational views:

- **1. Executive Business Dashboard (`/dashboard`):** Features four high-visibility metric summary cards displaying Today's Revenue, Monthly Sales Volume, Low Stock Alerts, and Gross Profit Margin. Incorporates interactive sales velocity trend lines (Recharts) and top-product stock tables.
- **2. Inventory & Stock Management View (`/inventory`):** Presents a searchable, paginated data grid of all catalog products with real-time stock counts, category tags, cost vs. selling prices, and color-coded status badges (Green = In Stock, Amber = Low Stock, Red = Out of Stock). Includes a modal for product creation/editing and manual stock adjustments.
- **3. Sales Terminal & Invoicing View (`/sales`):** Optimized for rapid checkout. Features product search, quantity selectors, dynamic line-item calculation, customer selection/guest toggle, tax rate input, discount controls, and an immediate receipt generator.
- **4. Procurement & Purchases View (`/purchases`):** Supplier purchase order generation interface, Goods Received Notes (GRN) logging, line-item cost tracking, and chronological purchase order history.
- **5. Customer Relationship Management (`/customers`):** Customer directory with live search by name/email, lifetime spend metrics, contact information editing, and historical order inspection drawers.
- **6. Financial Health & Accounting Ledger (`/finance`):** Restricted to ADMIN and OWNER. Displays revenue breakdown, operational expense tallies, net profit calculation, receivables, payables, and detailed financial transaction ledgers.
- **7. Workforce & Human Resources (`/employees`):** Staff roster, departmental allocations, operational role badges, monthly salary baselines, and employee provisioning modals.
- **8. Business Intelligence & Reports View (`/reports`):** Customizable date-range report generator with category filtering, automated alert feed (Critical/Warning), and one-click CSV and JSON data export buttons.

4.2 Hardware Interfaces

MSME360 does not require proprietary embedded hardware. It interfaces with standard commercial off-the-shelf (COTS) retail hardware devices via standard operating system drivers:

- **Workstation Hardware:** Desktop PCs, laptops, and tablets equipped with minimum 1024x768 display resolution and standard input peripherals (mouse/touchpad/keyboard).
- **Barcode Scanners:** Standard USB or Bluetooth barcode scanners operating in Human Interface Device (HID) keyboard emulation mode. Scanners input SKU strings directly into active search and sales item text fields.
- **Receipt & Invoice Printers:** Standard thermal receipt printers (58mm/80mm ESC/POS compatible) and standard office laser/inkjet printers via standard browser print dialogues (`window.print()`).

4.3 Software Interfaces & Data Exchange

MSME360 establishes well-defined software interfaces across internal application tiers and external host infrastructure:

- **Frontend-Backend REST Interface:** The React 18 client interfaces with the backend via stateless RESTful JSON APIs adhering to OpenAPI 3.1 specifications. All requests include standard HTTP headers (`Content-Type: application/json`, `Authorization: Bearer <jwt>`).
- **Database Engine Interface:** The FastAPI backend interfaces with PostgreSQL 15+ using the SQLAlchemy 2.0 Object-Relational Mapper over asynchronous connection pools. Schema migrations are strictly governed by Alembic migration scripts.
- **Containerization & Cloud Hosting:** The entire application is containerized using multi-stage Dockerfiles. The backend targets Linux-based container hosts on Render Cloud Web Services; static frontend assets are served via Vercel Edge CDN.

4.4 Communication Interfaces & Protocols

All data communication across client and server boundaries is governed by industry-standard networking protocols:

- **Transport Layer Security:** All external HTTP traffic is encrypted using Transport Layer Security (TLS 1.3 / HTTPS). Insecure HTTP requests are automatically upgraded or rejected.
- **CORS Governance:** Cross-Origin Resource Sharing (CORS) middleware is configured on the FastAPI server to whitelist authorized frontend origins while blocking unauthorized third-party domains.
- **Message Formatting:** All request and response bodies utilize UTF-8 encoded JSON formatting, with strict Pydantic schema validation preventing malformed payloads from reaching internal business logic.

5. Non-Functional Requirements (System Quality Attributes)

This section establishes the non-functional requirements (NFRs) governing the operational quality, performance, security, and maintainability of MSME360. Each requirement is defined with verifiable quantitative acceptance criteria and assigned a stable identifier (NFR-001 through NFR-020).

5.1 Performance, Latency, and Throughput

NFR ID	Quality Dimension	Requirement Statement & Quantitative Acceptance Criteria	Verification Method
NFR-001	API Response Latency	The backend API shall respond to 95% of standard read and write requests (excluding complex report exports) within 300 milliseconds under normal operating load.	Automated Load Testing (Locust)
NFR-002	Concurrent User Capacity	The system shall maintain full transactional integrity and sub-second response times under a minimum concurrent load of 50 active users.	Stress / Concurrency Testing
NFR-003	Data Volume Scalability	The relational database schema and query indexes shall support an operational catalog of at least 50,000 active SKUs and 500,000 transaction records without degrading query latency beyond 500ms.	Database Benchmark Script

5.2 Security, Cryptography, and Access Governance

NFR ID	Quality Dimension	Requirement Statement & Quantitative Acceptance Criteria	Verification Method
NFR-004	Password Encryption	The system shall store all user credentials hashed using bcrypt or Argon2 with an algorithmic work factor of at least 12. Plaintext passwords shall never be logged or persisted.	Code Inspection / Test
NFR-005	Session Security	All authenticated API transactions shall require a cryptographically signed JWT bearer token. Tokens shall expire exactly 24 hours after issuance and reject modified claims.	Automated Security Test
NFR-006	Injection Defense	The application shall utilize parameterized ORM queries (SQLAlchemy) exclusively, ensuring zero vulnerability to SQL injection attacks across all database inputs.	Static Analysis / DAST
NFR-007	XSS Mitigation	The frontend application shall sanitize all rendered user inputs through React JSX automatic context-aware escaping, preventing Cross-Site Scripting (XSS).	Security Code Review

5.3 Data Privacy and Confidentiality

NFR ID	Quality Dimension	Requirement Statement & Quantitative Acceptance Criteria	Verification Method
NFR-008	PII Protection	Customer phone numbers, personal email addresses, and employee salary records shall be restricted to authorized roles and masked in general application logs.	Log Inspection / Audit
NFR-009	Tenant Isolation	The database architecture shall enforce strict tenant isolation via indexed tenant_id foreign keys, preventing cross-tenant data leakage in multi-tenant deployments.	Integration Test Suite

5.4 Reliability, Fault Tolerance, and ACID Guarantees

NFR ID	Quality Dimension	Requirement Statement & Quantitative Acceptance Criteria	Verification Method
NFR-010	System Availability	The deployed cloud service shall maintain a minimum operational uptime of 99.5% during standard business operating hours (08:00 to 20:00 local time).	Cloud Uptime Monitoring
NFR-011	ACID Consistency	All multi-entity operations (e.g., sale completion + stock decrement + financial ledger entry) shall execute inside atomic database transactions; partial failures shall trigger complete rollback.	Fault Injection Testing

5.5 Usability, Human Factors, and Accessibility

NFR ID	Quality Dimension	Requirement Statement & Quantitative Acceptance Criteria	Verification Method
NFR-012	Checkout Efficiency	The Point-of-Sale interface shall enable a trained sales staff member to finalize a standard 3-item sale and generate an invoice in less than 30 seconds.	User Usability Evaluation
NFR-013	Visual Accessibility	The web application interface shall maintain a minimum color contrast ratio of 4.5:1 for standard text across all views, complying with WCAG 2.1 Level AA.	Automated Lighthouse Audit

5.6 Maintainability, Modularity, and Code Health

NFR ID	Quality Dimension	Requirement Statement & Quantitative Acceptance Criteria	Verification Method
NFR-014	Architectural Modularity	Backend services, database repositories, and API routers shall be decoupled such that individual operational modules can be modified without side effects.	Architectural Code Review
NFR-015	Test Suite Coverage	The automated unit and integration test suite (Pytest) shall achieve a minimum of 80% line coverage across all backend business logic and service classes.	Pytest-Cov Report

5.7 Portability and Cross-Platform Compatibility

NFR ID	Quality Dimension	Requirement Statement & Quantitative Acceptance Criteria	Verification Method
NFR-016	Browser Compatibility	The frontend application shall render and operate identically without functional defects across modern evergreen browsers: Chrome, Firefox, Edge, Safari.	Cross-Browser Verification
NFR-017	Responsive Layout	The interface shall adapt fluidly across viewport widths ranging from 1024px (tablet landscape) to 3840px (4K ultra-wide) using responsive CSS grids.	Responsive UI Inspection

5.8 Scalability and Resource Utilization

NFR ID	Quality Dimension	Requirement Statement & Quantitative Acceptance Criteria	Verification Method

NFR ID	Quality Dimension	Requirement Statement & Quantitative Acceptance Criteria	Verification Method
NFR-018	Resource Footprint	The backend application container shall consume less than 256 MB of RAM in an idle state and not exceed 512 MB under standard concurrent user load.	Docker Stats / Monitoring

5.9 Diagnostics, Auditing, and Error Handling

NFR ID	Quality Dimension	Requirement Statement & Quantitative Acceptance Criteria	Verification Method
NFR-019	Structured Error Responses	All API exceptions shall return standardized JSON error objects containing `success: false`, human-readable `message`, and structured error codes.	Automated API Exception Test
NFR-020	Health Check Diagnostic	The backend shall expose an unauthenticated health check endpoint `/api/health` returning system operational status and service identifier in < 50ms.	Uptime Monitoring Probe

6. Data Requirements and Schema Architecture

6.1 Logical Data Model & Entity Relationship Architecture

The MSME360 relational database architecture is strictly normalized to Third Normal Form (3NF) to eliminate data redundancy and prevent update anomalies across high-frequency transactional workflows. Referential integrity is maintained through foreign key constraints with explicit cascading delete or set-null rules.

Core Relational Entities: The logical entity structure comprises fourteen core domain entities:

- **Users & Roles:** Defines system operators, contact emails, cryptographic password hashes, and assigned operational roles.
- **Categories & Products:** Hierarchical taxonomy groups governing individual inventory items.
- **Stock Movements:** Immutable chronological audit log capturing every quantity delta across purchases, sales, adjustments, and returns.
- **Customers & Suppliers:** Commercial client profiles and procurement vendor registries.
- **Purchases & Sales (Orders & Items):** Master transaction records and normalized line-item child tables detailing quantities, unit prices, and subtotals.
- **Financial Transactions:** General financial ledger recording all atomic income and expense entries tied to business receipts.
- **Employees & System Alerts:** Internal workforce roster and dynamic inventory warning notices.

6.2 Data Dictionaries and Schema Specifications

Tables 6.1 through 6.6 specify the physical schema definitions, column constraints, data types, and indexed keys for the primary database tables.

Entity Specification: Table 6.1: Data Dictionary — Users (`users`)

Column Name	Data Type	Nullability	Key / Index	Description & Constraints
id	VARCHAR(36)	NOT NULL	PK	Unique surrogate identifier (UUIDv4).
username	VARCHAR(50)	NOT NULL	UNIQUE, B-tree	Unique login credential identifier.
email	VARCHAR(100)	NOT NULL	UNIQUE	Verified corporate email address.
password_hash	VARCHAR(255)	NOT NULL	None	Cryptographic bcrypt password hash.
name	VARCHAR(100)	NOT NULL	None	Full legal name of the user.
role	VARCHAR(20)	NOT NULL	FK -> roles.name	Assigned RBAC role (ADMIN, OWNER, MANAGER, STAFF).
is_active	BOOLEAN	NOT NULL	None	Account status flag; defaults to True.
created_at	TIMESTAMP	NOT NULL	None	UTC creation timestamp; defaults to `now()`.

Entity Specification: Table 6.2: Data Dictionary — Products (`products`)

Column Name	Data Type	Nullability	Key / Index	Description & Constraints

Column Name	Data Type	Nullability	Key / Index	Description & Constraints
id	VARCHAR(36)	NOT NULL	PK	Unique product identifier (UUIDv4).
name	VARCHAR(150)	NOT NULL	B-tree	Commercial name of the inventory product.
sku	VARCHAR(50)	NOT NULL	UNIQUE, B-tree	Unique alphanumeric Stock Keeping Unit.
category_id	VARCHAR(36)	NOT NULL	FK -> categories.id	Parent category foreign key.
cost_price	NUMERIC(12,2)	NOT NULL	None	Procurement unit acquisition cost (CHECK > 0).
selling_price	NUMERIC(12,2)	NOT NULL	None	Retail selling price (CHECK > 0).
stock_quantity	INTEGER	NOT NULL	B-tree	Current physical inventory on hand (CHECK >= 0).
min_stock_level	INTEGER	NOT NULL	None	Threshold triggering LOW_STOCK alert; defaults to 5.
is_active	BOOLEAN	NOT NULL	None	Catalog availability flag; defaults to True.

Entity Specification: Table 6.3: Data Dictionary — Stock Movements (`stock_movements`)

Column Name	Data Type	Nullability	Key / Index	Description & Constraints
id	VARCHAR(36)	NOT NULL	PK	Unique stock movement ledger identifier.
product_id	VARCHAR(36)	NOT NULL	FK -> products.id	Associated inventory item foreign key.
movement_type	VARCHAR(20)	NOT NULL	B-tree	Type: PURCHASE, SALE, ADJUSTMENT, RETURN.
quantity	INTEGER	NOT NULL	None	Signed quantity delta (positive or negative).
previous_stock	INTEGER	NOT NULL	None	Stock balance immediately prior to movement.
new_stock	INTEGER	NOT NULL	None	Stock balance immediately following movement.
reason	TEXT	NOT NULL	None	Mandatory audit reason or order reference string.
timestamp	TIMESTAMP	NOT NULL	B-tree	UTC timestamp of the stock event.

Entity Specification: Table 6.4: Data Dictionary — Sales Orders (`sales`)

Column Name	Data Type	Nullability	Key / Index	Description & Constraints
id	VARCHAR(36)	NOT NULL	PK	Unique sales invoice identifier.
customer_id	VARCHAR(36)	NULL	FK -> cust.id	Purchasing customer ID; NULL for guest checkouts.
customer_name	VARCHAR(100)	NOT NULL	None	Customer name or 'Guest Customer'.
subtotal	NUMERIC(12,2)	NOT NULL	None	Aggregate line items subtotal.

Column Name	Data Type	Nullability	Key / Index	Description & Constraints
tax	NUMERIC(12,2)	NOT NULL	None	Computed tax amount.
discount	NUMERIC(12,2)	NOT NULL	None	Applied discount deduction.
total	NUMERIC(12,2)	NOT NULL	B-tree	Net payable invoice total (CHECK >= 0).
status	VARCHAR(20)	NOT NULL	B-tree	Order status: COMPLETED, CANCELLED.
created_at	TIMESTAMP	NOT NULL	B-tree	UTC timestamp of order execution.

Entity Specification: Table 6.5: Data Dictionary — Financial Transactions (`financial_transactions`)

Column Name	Data Type	Nullability	Key / Index	Description & Constraints
id	VARCHAR(36)	NOT NULL	PK	Unique financial transaction identifier.
type	VARCHAR(10)	NOT NULL	B-tree	Ledger classification: INCOME, EXPENSE.
category	VARCHAR(50)	NOT NULL	B-tree	Category: SALES, PURCHASES, PAYROLL, UTILITIES.
amount	NUMERIC(12,2)	NOT NULL	None	Monetary transaction magnitude (CHECK > 0).
reference_id	VARCHAR(50)	NULL	B-tree	Originating sale ID, purchase ID, or expense receipt.
description	TEXT	NOT NULL	None	Narrative description of the financial transaction.
created_at	TIMESTAMP	NOT NULL	B-tree	UTC timestamp of entry creation.

Entity Specification: Table 6.6: Data Dictionary — System Alerts (`alerts`)

Column Name	Data Type	Nullability	Key / Index	Description & Constraints
id	VARCHAR(36)	NOT NULL	PK	Unique system alert identifier.
type	VARCHAR(30)	NOT NULL	B-tree	Alert type: LOW_STOCK, OUT_OF_STOCK, SALES_DROP.
severity	VARCHAR(15)	NOT NULL	B-tree	Severity level: INFO, WARNING, CRITICAL.
title	VARCHAR(100)	NOT NULL	None	Brief alert headline.
message	TEXT	NOT NULL	None	Detailed diagnostic message and contextual metrics.
is_read	BOOLEAN	NOT NULL	None	Acknowledgment flag; defaults to False.
created_at	TIMESTAMP	NOT NULL	B-tree	UTC timestamp of alert trigger.

6.3 Data Validation Rules and Integrity Constraints

The database layer and Pydantic validation schemas enforce rigid operational integrity rules:

- **Non-Negative Bounds:** All monetary prices (`cost_price`, `selling_price`), financial transaction amounts, and employee salaries must be strictly positive numerical floats (`value > 0`). Line item quantities must be positive integers (`quantity > 0`).
- **Stock Floor Enforcement:** Physical inventory balances (`stock_quantity`) cannot fall below zero under any operational circumstance. Any transaction attempting to decrement stock below zero is aborted immediately.
- **Cascading Referential Integrity:** Foreign keys from child tables (e.g., `sale_items.product_id`) reference valid parent records. Deleting a parent category or product with active transaction history is prohibited (RESTRICT) to prevent orphaned historical records.
- **Indexing Strategy:** Composite B-tree indexes are deployed across `(tenant_id, created_at, sku)` to optimize range queries, sales filtering, and real-time dashboard aggregations.

6.4 Storage, Retention, and Archival Guidelines

Operational data retention protocols ensure audit compliance and database longevity:

- **Immutable Financial Records:** Sales orders, purchase receipts, stock movements, and financial ledger entries are preserved indefinitely as permanent immutable business records.
- **Soft Deletion Strategy:** Products, categories, customers, and employees utilize soft deactivation (`is_active = False`) rather than physical deletion (`DELETE`), ensuring historical sales invoices resolve accurately.
- **Backup Cadence:** Automated database backups (full snapshots daily, Write-Ahead Log WAL archiving hourly) are maintained with a rolling 30-day recovery window.

6.5 Data Import and Export Formats

MSME360 supports structured data interchange via standard CSV (RFC 4180) and JSON (RFC 8259) specifications. Exported sales and inventory reports include metadata headers, ISO-8601 UTC timestamps, and normalized numerical columns formatted for immediate analysis in external spreadsheet software.

7. Operational Use Cases and System Workflows

This section documents eight critical end-to-end use cases illustrating the primary operational interactions within MSME360. Each use case is formally structured with participating actors, preconditions, triggers, main success scenarios, alternative flows, exception handling, and related functional requirement mappings.

UC-001: User Authentication and Role-Based Session Initiation

Use Case Attribute	Operational Specification
Primary Actor	Any System User (ADMIN, OWNER, MANAGER, STAFF)
Preconditions	User has an active account with registered credentials.
Trigger	User accesses web application and submits login form.

Use Case Attribute	Operational Specification
Main Success Scenario	1. User enters username and password. 2. Client POSTs credentials to `/api/auth/login`. 3. Backend queries user record and verifies password hash using bcrypt. 4. Backend generates signed JWT access token containing role claims. 5. Backend returns token and user profile. 6. Frontend stores token, updates authentication state, and renders role-appropriate dashboard.
Alternative Flows	A1: User already holds valid token in local storage; client auto-authenticates via `/api/auth/me`.
Exception Flows	E1: Invalid credentials → System displays error 'Invalid username or password' (401). E2: Deactivated account → System displays 'Account disabled' (403).
Postconditions	Authenticated session active; token injected into all subsequent HTTP requests.
Related Functional Reqs	FR-001, FR-002, FR-003, FR-004, FR-005

UC-002: Product Catalog Setup and Reorder Threshold Configuration

Use Case Attribute	Operational Specification
Primary Actor	Store Manager (MANAGER), Business Owner (OWNER), Admin (ADMIN)
Preconditions	Target category exists. User holds MANAGER, OWNER, or ADMIN role.
Trigger	User clicks '+ Add Product' button on the Inventory page.
Main Success Scenario	1. User fills product form: Name, unique SKU, Category, Cost Price, Selling Price, Initial Stock, and Min Stock Level. 2. User submits form. 3. Frontend sends POST `/api/inventory/products`. 4. Backend validates SKU uniqueness and positive pricing. 5. Backend creates Product record and logs initial ADJUSTMENT stock movement. 6. Backend returns 201 Created. 7. UI appends new product to inventory data grid.
Alternative Flows	A1: User edits existing product thresholds via PUT `/api/inventory/products/{id}`.
Exception Flows	E1: Duplicate SKU code entered → System returns 400 Bad Request with code `DUPLICATE_SKU`. E2: Zero or negative price entered → System rejects with 422 Unprocessable Entity.
Postconditions	Product is active in catalog with assigned reorder threshold and inventory balance.
Related Functional Reqs	FR-007, FR-008, FR-009, FR-010, FR-011, FR-012

UC-003: Supplier Purchase Order Execution and Stock Replenishment

Use Case Attribute	Operational Specification

Use Case Attribute	Operational Specification
Primary Actor	Store Manager (MANAGER), Business Owner (OWNER)
Preconditions	Supplier is identified; products exist in catalog.
Trigger	User clicks 'New Purchase Order' on the Purchases page.
Main Success Scenario	1. User enters supplier name, selects products, enters quantities, unit costs, tax rate, and discount. 2. User submits purchase order. 3. Backend validates line items and calculates net total. 4. Backend creates Purchase record (`status = COMPLETED`). 5. Backend atomically increments product stock quantities. 6. Backend logs PURCHASE records in stock movements ledger. 7. Backend inserts EXPENSE transaction in general financial ledger. 8. UI displays confirmation and updates stock quantities.
Alternative Flows	A1: Business Owner cancels purchase order via `/api/business/purchases/{id}/cancel`; stock is decremented and order marked CANCELLED.
Exception Flows	E1: Empty item list submitted → System returns 400 `EMPTY_ITEMS`. E2: Cancellation attempted on already cancelled order → System returns 400 `ALREADY_CANCELLED`.
Postconditions	Stock balances increased atomically; procurement expenditure posted to general ledger.
Related Functional Reqs	FR-019, FR-020, FR-021, FR-022, FR-023, FR-024

UC-004: Point-of-Sale (POS) Checkout with Atomic Stock Deduction

Use Case Attribute	Operational Specification
Primary Actor	Sales Staff (STAFF), Store Manager (MANAGER), Owner (OWNER)
Preconditions	Sufficient product stock is available in inventory.
Trigger	Sales operator clicks 'Complete Sale' at the sales terminal.
Main Success Scenario	1. Operator selects or registers customer (or toggles Guest mode). 2. Operator adds products and quantities to cart. 3. Operator enters tax rate and applicable discount. 4. Operator clicks 'Complete Sale'. 5. Backend verifies stock availability for all items. 6. Backend creates Sale record (`status = COMPLETED`). 7. Backend atomically decrements stock for each item. 8. Backend records SALE audit entries in stock movements table. 9. Backend creates INCOME transaction in general ledger. 10. If registered customer, customer's `total_spent` is updated. 11. Frontend renders printable sales receipt.
Alternative Flows	A1: Guest sale executed without customer ID; invoice assigned to 'Guest Customer'.

Use Case Attribute	Operational Specification
Exception Flows	E1: Stock requested exceeds available inventory → Backend aborts transaction and returns 400 `INSUFFICIENT_STOCK` detailing requested vs available quantity. E2: Invalid discount exceeding subtotal → System returns 422.
Postconditions	Sale finalized, stock decremented, revenue logged, customer spend updated.
Related Functional Reqs	FR-025, FR-026, FR-027, FR-028, FR-029, FR-031

UC-005: Physical Inventory Count Audit and Stock Adjustment

Use Case Attribute	Operational Specification
Primary Actor	Store Manager (MANAGER), Business Owner (OWNER)
Preconditions	Product exists and discrepancy is identified during warehouse audit.
Trigger	User clicks 'Adjust Stock' on a specific inventory row.
Main Success Scenario	1. User specifies quantity change (e.g., -2 for damaged goods, +5 for found stock). 2. User selects movement type (`ADJUSTMENT` or `RETURN`) and inputs mandatory audit reason. 3. Frontend sends POST `/api/inventory/products/{id}/adjust-stock`. 4. Backend validates resulting stock (`stock_quantity + change >= 0`). 5. Backend updates product stock balance. 6. Backend appends audit entry to `stock_movements` table. 7. Backend re-evaluates low stock status and returns updated product.
Alternative Flows	A1: Stock increment resulting from customer return.
Exception Flows	E1: Deduction would cause negative stock balance → Backend rejects with 400 `INSUFFICIENT_STOCK`. E2: Empty reason string → Backend rejects with 422.
Postconditions	Inventory balance reconciled; immutable audit trail preserved.
Related Functional Reqs	FR-013, FR-014, FR-015, FR-016, FR-018

UC-006: Sales Order Cancellation and Transaction Reversal

Use Case Attribute	Operational Specification
Primary Actor	Store Manager (MANAGER), Business Owner (OWNER)
Preconditions	Sale is in COMPLETED status.
Trigger	Customer requests full refund / order cancellation.

Use Case Attribute	Operational Specification
Main Success Scenario	1. User navigates to Sales history and selects sale invoice. 2. User clicks 'Cancel Sale' and confirms dialog. 3. Frontend sends POST `/api/business/sales/{id}/cancel`. 4. Backend verifies sale is currently COMPLETED. 5. Backend updates sale status to CANCELLED. 6. Backend restores item quantities to inventory via `RETURN` movement records. 7. Backend decrements customer's cumulative `total_spent`. 8. Backend returns updated sale payload. 9. UI displays cancelled status badge.
Alternative Flows	None.
Exception Flows	E1: Sale is already in CANCELLED status → Backend returns 400 `ALREADY_CANCELLED`. E2: Unauthorized role (STAFF) attempts cancellation → Backend returns 403 Forbidden.
Postconditions	Sale cancelled, inventory restored, customer spend adjusted.
Related Functional Reqs	FR-014, FR-029, FR-030

UC-007: Financial Health Review and Debtor Aging Inspection

Use Case Attribute	Operational Specification
Primary Actor	Business Owner (OWNER), System Admin (ADMIN)
Preconditions	User authenticated with OWNER or ADMIN role.
Trigger	User navigates to Finance tab (`/finance`).
Main Success Scenario	1. Frontend sends GET `/api/finance/summary` and GET `/api/finance/transactions`. 2. Backend aggregates sales revenue, purchase expenses, and operational overheads. 3. Backend computes gross profit, net profit, and profit margin percentage. 4. Backend aggregates total receivables and payables. 5. Backend returns financial summary and transaction list. 6. UI renders metric cards (Total Revenue, Expenses, Net Profit, Margin) and transaction table.
Alternative Flows	A1: User filters transactions by date range or transaction category.
Exception Flows	E1: User with MANAGER or STAFF role attempts navigation → System denies access with 403 Forbidden.
Postconditions	Financial performance metrics visualized for strategic decision making.
Related Functional Reqs	FR-037, FR-038, FR-039, FR-040, FR-041, FR-042

UC-008: Automated Stock Threshold Alert Dispatch and Report Export

Use Case Attribute	Operational Specification

Use Case Attribute	Operational Specification
Primary Actor	Store Manager (MANAGER), Business Owner (OWNER)
Preconditions	One or more products have fallen below their minimum stock thresholds.
Trigger	User accesses Reports view (`/reports`) or Alert notification center.
Main Success Scenario	1. Frontend sends GET `/api/alerts`. 2. Backend evaluates all products against `min_stock_level` and zero stock. 3. Backend compiles active alerts (CRITICAL for zero stock, WARNING for low stock). 4. User reviews alert feed and selects 'Sales & Revenue Summary' report. 5. User specifies date range filter and clicks 'Export CSV'. 6. Frontend sends POST `/api/reports/sales` with filter parameters. 7. Backend generates report dataset and returns structured rows. 8. Frontend converts dataset to CSV file and triggers browser download.
Alternative Flows	A1: User exports data in JSON format for external programmatic processing.
Exception Flows	E1: Invalid date filter string → Backend returns 422 Unprocessable Entity. E2: No matching transaction data for selected date range → System returns empty report with zero totals.
Postconditions	Alerts acknowledged; formatted business intelligence report downloaded to local workstation.
Related Functional Reqs	FR-048, FR-049, FR-050, FR-051, FR-052

8. System Constraints, Assumptions, and Limitations

8.1 Mandatory Technical & Architectural Constraints

The technical architecture of MSME360 is bound by non-negotiable engineering constraints:

- **Framework Constraint:** The backend must be implemented using Python 3.12+ and the FastAPI asynchronous framework, exposing stateless RESTful JSON endpoints.
- **Relational Persistence Constraint:** The target production database must be PostgreSQL 15+, normalized to Third Normal Form (3NF) and managed via Alembic version-controlled migrations.
- **Frontend Presentation Constraint:** The frontend must be constructed as a Single Page Application (SPA) using React 18, Vite build tooling, and Tailwind CSS for utility-first styling.
- **Cryptographic Protocol Constraint:** All user authentication must utilize standard OAuth2 Bearer password flow issuing signed JSON Web Tokens (JWT) using the HS256 algorithm.
- **Academic Milestone Alignment:** The architecture must strictly align with the academic syllabus across Database Management Systems (DBMS), Design and Analysis of Algorithms (DAA), Software Engineering (SE), and Web Technology (WT).

8.2 Validated Engineering Assumptions

The functional design assumes the following validated environmental parameters:

- **Operational Connectivity:** Network latency between client browsers and backend cloud services does not exceed 250ms under normal operating conditions.
- **Client Runtime Environment:** Client workstations operate modern evergreen web browsers with full ES2022 JavaScript, LocalStorage, and CSS Flexbox/Grid support enabled.
- **User Operational Security:** Users adhere strictly to credential confidentiality protocols; session tokens are stored securely in browser session contexts.

8.3 Known Implementation Limitations of the Current Baseline

To ensure academic transparency and implementation clarity, known limitations of the Version 1.0 baseline are documented:

- **1. In-Memory Prototype Persistence:** The prototype utilizes an in-memory repository layer for instantaneous test execution and local demonstration. While functionally identical in business logic and atomic behavior, enterprise deployment requires binding the SQLAlchemy PostgreSQL connection string.
- **2. Cash / External Payment Settlements:** All sales transactions record internal billing receipts and income ledgers, but direct external payment gateway capture (Razorpay/Stripe) is not executed in Version 1.0.
- **3. Online Cloud Connectivity Requirement:** The system requires active network connectivity; offline transactional queueing with client-side SQLite/IndexedDB synchronization is deferred to future releases.
- **4. Synchronous Alert Engine:** Inventory alerts are evaluated synchronously upon API request; an independent asynchronous background message queue (e.g., Celery/Redis) is targeted for Version 2.0.

9. Verification Strategy and Acceptance Criteria

9.1 Verification Methods and Quality Gates

Requirements verification is structured around four formal evaluation methods recognized in industrial systems engineering:

- **1. Inspection (I):** Visual or programmatic examination of software artifacts, source code syntax, database migration scripts, and documentation.
- **2. Demonstration (D):** Observational execution of user workflows through the graphical user interface to verify end-to-end functional behavior.
- **3. Analysis (A):** Quantitative mathematical evaluation of calculations, response latencies, memory footprint, and test code coverage metrics.
- **4. Testing (T):** Automated execution of deterministic test cases (unit, integration, security, boundary) using testing frameworks (Pytest, Postman).

9.2 Acceptance Criteria Matrix

Table 9.1 establishes fifteen mandatory acceptance criteria (AC-001 through AC-015) that the system must satisfy for final project sign-off.

AC ID	Target Capability	Mandatory Acceptance Criterion Scenario	Method
AC-001	Authentication	User submits valid credentials → System returns JWT access token within 300ms; invalid credentials return HTTP 401 with structured error code `INVALID_CREDENTIALS`.	T, D
AC-002	Role Authorization	STAFF user attempts to access `/api/finance/summary` → System aborts request and returns HTTP 403 Forbidden with role rejection message.	T
AC-003	SKU Uniqueness	Admin attempts to create product with existing SKU → System rejects insertion and returns HTTP 400 Bad Request with code `DUPLICATE_SKU`.	T
AC-004	Positive Pricing	User attempts to submit product with cost_price <= 0 or selling_price <= 0 → Pydantic validation rejects payload with HTTP 422.	T
AC-005	Stock Decrement	Completed sale of 3 units of SKU-A reduces available inventory from 40 to 37 and appends a SALE record to `stock_movements`.	T, D
AC-006	Insufficient Stock	User attempts to sell 10 units when only 5 units exist in stock → System rejects sale with HTTP 400 `INSUFFICIENT_STOCK` without altering inventory.	T
AC-007	Atomic Rollback	Simulated failure during sale finalization reverts stock decrement, cancels receipt, and logs zero financial ledger entries.	T, A

AC ID	Target Capability	Mandatory Acceptance Criterion Scenario	Method
AC-008	Purchase Restock	Completed purchase order for 20 units of SKU-B increases inventory balance from 10 to 30 and logs EXPENSE entry in general ledger.	T, D
AC-009	Customer Spend	Completing a $150 sale linked to Customer-1 increments Customer-1's `total_spent` attribute by exactly $150.00.	T
AC-010	Order Reversal	Cancelling an active sale returns items to inventory via RETURN movements and decrements customer lifetime spend.	T, D
AC-011	Financial Margins	Financial summary accurately computes gross profit, net profit, and profit margin percent matching analytical spreadsheet models within 0.01 precision.	A, T
AC-012	Stock Threshold Alert	Product with stock_quantity <= min_stock_level triggers a WARNING alert in `/api/alerts`; stock_quantity <= 0 triggers CRITICAL alert.	T, D
AC-013	Dynamic ROP Calc	Inventory analytics engine computes ROP matching formula: (Avg Daily Usage * Lead Time) + Safety Stock.	A, T
AC-014	Report Data Export	User requests sales report export → System generates valid CSV and JSON file downloads containing all matching transaction rows and summary totals.	D, I
AC-015	Test Suite Health	Automated Pytest test suite executes and passes 100% of integration test cases with zero runtime errors.	T, A

10. Requirements Traceability Matrix (RTM)

Table 10.1 establishes a comprehensive bidirectional Requirements Traceability Matrix (RTM), mapping core project objectives and stakeholder needs to system modules, functional requirements, quality attributes, operational use cases, acceptance criteria, and verified source materials.

Project Objective / Need	Module	Functional Reqs	Quality Attr	Use Case	Acceptance	Source Reference
OBJ-1: Secure Role-Based Access	M1: Auth & RBAC	FR-001 to FR-006	NFR-004, NFR-005	UC-001	AC-001, AC-002	Proposal PDF (p.1), Code (`auth.py`)
OBJ-2: Unified Product Catalog	M2: Catalog & SKU	FR-007 to FR-012	NFR-003, NFR-006	UC-002	AC-003, AC-004	Slide Deck (p.4, 7), Code (`inventory.py`)
OBJ-3: Real-Time Stock Auditing	M3: Stock Control	FR-013 to FR-018	NFR-010, NFR-011	UC-005	AC-005, AC-012	Proposal PDF (p.1), Code (`inventory_service.py`)

Project Objective / Need	Module	Functional Reqs	Quality Attr	Use Case	Acceptance	Source Reference
OBJ-4: Procurement & Restocking	M4: Procurement	FR-019 to FR-024	NFR-001, NFR-011	UC-003	AC-008	Slide Deck (p.4), Code (`purchase_service.py`)
OBJ-5: POS Checkout & Invoicing	M5: Sales POS	FR-025 to FR-031	NFR-001, NFR-012	UC-004, UC-006	AC-005, AC-006, AC-010	Slide Deck (p.4, 7), Code (`sale_service.py`)
OBJ-6: Customer CRM & Spend	M6: Customer CRM	FR-032 to FR-036	NFR-008, NFR-009	UC-004	AC-009	Slide Deck (p.4, 8), Code (`customer_service.py`)
OBJ-7: Financial Ledger & Profit	M7: Finance & P&L	FR-037 to FR-042	NFR-004, NFR-011	UC-007	AC-011	Slide Deck (p.4, 7), Code (`finance_service.py`)
OBJ-8: Workforce Directory	M8: Workforce HR	FR-043 to FR-046	NFR-008, NFR-014	UC-001	AC-002	Slide Deck (p.4, 8), Code (`employee_service.py`)
OBJ-9: Actionable BI & Analytics	M9: BI & Reporting	FR-047 to FR-052	NFR-001, NFR-013	UC-008	AC-013, AC-014	Proposal (p.1), Slides (p.7, 8), Code (`analytics...py`)
OBJ-10: Academic Software Health	Cross-Cutting	All FRs (001-052)	NFR-015 to NFR-020	All UCs	AC-015	Pytest Suite (`backend/tests`), Git Repository

11. Appendices

Appendix A: Comprehensive Technical Glossary

Table 11.1 provides an authoritative reference glossary of technical terms utilized throughout the specification.

Technical Term	Contextual Engineering Definition
Atomicity	Transactional property guaranteeing that a multi-step operation either completes entirely or leaves data unchanged with zero partial mutations.

Technical Term	Contextual Engineering Definition
B-tree Index	Balanced tree data structure utilized within PostgreSQL to maintain sorted keys and provide logarithmic time complexity O(log N) for queries.
Cascading Delete	Foreign key constraint rule automatically purging dependent child records when the referenced parent primary key is removed.
Dynamic ROP	Algorithmic procurement trigger dynamically recalculated based on actual rolling sales velocity, delivery lead time, and statistical buffer stock.
Goods Received Note (GRN)	Formal receipt acknowledging delivery of physical goods matching an approved supplier purchase order.
JWT Bearer Token	Cryptographically signed compact token passed in HTTP Authorization headers to verify authenticated user identity statelessly.
MoSCoW Method	Requirements prioritization technique classifying requirements into Must have, Should have, Could have, and Won't have categories.
Pydantic	Data parsing and validation library for Python utilizing type annotations to enforce strict runtime schema compliance.
Single Page Application	Web architecture executing dynamic view changes within the browser without requiring full-page server reloads.
TreeSHAP	Explainability algorithm referenced in historical benchmark documents, utilized for evaluating decision tree feature attributions.

Appendix B: MoSCoW Prioritization Framework

Prioritization Criteria: Requirements within this SRS are prioritized utilizing the industrial MoSCoW framework:

- **Must Have (M):** Non-negotiable functional capabilities mandatory for system operation, security, and academic baseline evaluation (FR-001 to FR-022, FR-025 to FR-029, FR-032 to FR-035, FR-037 to FR-041, FR-043 to FR-045, FR-047 to FR-052).
- **Should Have (S):** Highly desirable capabilities that significantly enhance operational efficiency but do not prevent baseline execution (FR-023, FR-024, FR-030, FR-031, FR-036, FR-042, FR-046).
- **Could Have (C):** Enhancement capabilities targeted for deployment if development velocity permits (e.g., custom report builder templates, dark mode interface toggle).
- **Won't Have (W - Version 1.0):** Explicitly excluded capabilities reserved for future major release roadmaps (e.g., direct banking payment gateways, live GSTN government portal e-filing).

Appendix C: Assumptions and Open Issues Register

Table 11.2 logs technical assumptions, resolved documentation conflicts, and open decisions requiring academic or operational confirmation.

Issue ID	Category	Description of Discrepancy or Decision	Status / Architectural Resolution

Issue ID	Category	Description of Discrepancy or Decision	Status / Architectural Resolution
ISSUE-01	Architecture Stack Conflict	Slide 9 of project presentation ('SaaS Business Suites.pdf') mentions 'Spring Boot REST APIs' and 'Spring Data IPA', conflicting with Proposal PDF ('Python 3.12, FastAPI, SQLAlchemy') and working codebase.	RESOLVED: Authoritative stack is confirmed as Python 3.12 / FastAPI. The presentation slide contained a legacy template copy-paste artifact.
ISSUE-02	Database Engine Binding	The verified implementation currently executes on an in-memory repository for rapid test execution and local workstation independence.	OPEN FOR MILESTONE-2: PostgreSQL database connection string and Alembic migrations to be finalized for multi-user deployment.
ISSUE-03	Government Taxation Integration	Stakeholders inquired whether live GST / e-Way bill submission should be integrated into Version 1.0.	RESOLVED: Strictly excluded from Version 1.0 scope due to regulatory credential requirements. Scheduled for Version 2.0 roadmap.
ISSUE-04	Payment Gateway Capture	Evaluation of commercial payment gateway webhooks (Razorpay / Stripe) for automated sales settlement.	RESOLVED: Excluded from current release. All payments recorded as internal ledger receipts (Cash, Bank Transfer, Offline Credit).

Appendix D: Complete REST API Endpoint Inventory

Table 11.3 documents the complete catalog of verified REST API endpoints exposed by the MSME360 FastAPI application server.

HTTP Verb	API Endpoint Route	Allowed Roles	Summary & Description
POST	/api/auth/login	Public / All	Authenticates user credentials and issues 24-hour signed JWT access token.
GET	/api/auth/me	All Authenticated	Retrieves current authenticated user profile, permissions, and assigned role.
GET	/api/health	Public / All	Unauthenticated diagnostic health check probe returning service status.
GET	/api/dashboard/summary	All Authenticated	Aggregates daily sales, monthly revenue, low-stock count, and top products.
GET	/api/inventory/categories	All Authenticated	Retrieves all product category taxonomy records.

HTTP Verb	API Endpoint Route	Allowed Roles	Summary & Description
GET	/api/inventory/products	All Authenticated	Lists and searches products by name, SKU, or category with stock status.
GET	/api/inventory/products/{id}	All Authenticated	Retrieves detailed single product profile, pricing, and thresholds.
POST	/api/inventory/products	ADMIN, OWNER, MANAGER	Creates new catalog product with initial stock balance and SKU.
PUT	/api/inventory/products/{id}	ADMIN, OWNER, MANAGER	Updates product name, pricing, category, or min_stock_level.
POST	/api/inventory/products/{id}/adjust-stock	ADMIN, OWNER, MANAGER	Executes manual stock adjustment with mandatory audit reason capture.
GET	/api/inventory/movements	All Authenticated	Returns chronological immutable audit ledger of all stock movements.
GET	/api/business/customers	All Authenticated	Lists customers with optional substring search and total_spent balances.
GET	/api/business/customers/{id}	All Authenticated	Retrieves single customer profile and purchasing history.
POST	/api/business/customers	ADMIN, OWNER, MANAGER, STAFF	Registers new customer profile with contact information.
PUT	/api/business/customers/{id}	ADMIN, OWNER, MANAGER	Updates existing customer contact email and phone details.
GET	/api/business/purchases	All Authenticated	Lists all supplier purchase orders with line items and totals.
POST	/api/business/purchases	ADMIN, OWNER, MANAGER	Creates purchase order, restocks inventory, and logs EXPENSE ledger entry.
POST	/api/business/purchases/{id}/cancel	ADMIN, OWNER	Cancels purchase order and reverses stock increments.
GET	/api/business/sales	All Authenticated	Lists all customer sales transactions and order statuses.
GET	/api/business/sales/{id}	All Authenticated	Retrieves detailed itemized sales invoice.
POST	/api/business/sales	ADMIN, OWNER, MANAGER, STAFF	Executes POS sale, decrements stock atomically, logs INCOME transaction.
POST	/api/business/sales/{id}/cancel	ADMIN, OWNER, MANAGER	Cancels completed sale, returns stock via RETURN, decrements customer spend.

HTTP Verb	API Endpoint Route	Allowed Roles	Summary & Description
GET	/api/finance/summary	ADMIN, OWNER	Computes total revenue, expenses, gross profit, net profit, and profit margin.
GET	/api/finance/transactions	ADMIN, OWNER	Returns complete chronological general financial transaction ledger.
GET	/api/business/employees	ADMIN, OWNER, MANAGER	Lists active employee roster with departments and base salaries.
POST	/api/business/employees	ADMIN, OWNER	Creates new employee record with role, department, and salary.
GET	/api/analytics/sales	All Authenticated	Returns historical daily sales trends and top-selling product statistics.
GET	/api/analytics/inventory	All Authenticated	Returns stock turnover velocity, low-stock frequency, and category distribution.
GET	/api/alerts	All Authenticated	Evaluates and returns dynamic inventory threshold alerts (LOW_STOCK, OUT_OF_STOCK).
POST	/api/reports/sales	All Authenticated	Generates comprehensive filtered sales report for CSV/JSON export.

--- END OF SOFTWARE REQUIREMENTS SPECIFICATION (MSME360) ---